

DATA-260 Homework 1 – Report

Uday Patel

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16” (Ryzen 5 7640HS, 16GB RAM, NVIDIA RTX 4060 8GB, 512GB SSD)

Local model used: qwen3:8b, served via Ollama

Repository: <https://github.com/UdayPate/data260-9871>

Tagged commit: hwl – 84e01fdcce624279d0fc58174af0c6af4910425a
<https://github.com/UdayPate/data260-9871/tree/84e01fdcce624279d0fc58174af0c6af4910425a>

Part 1 – HTML, JavaScript, Docker, AWS ECS

A Community Sports League Fixtures form (`index.html`) was built with a required primary field (fixture name, autofocus), secondary field (teams/players), submitter email, description textarea, sport-type dropdown, and a terms-and-conditions checkbox. `app.js` implements arrow-function validation (description >25 characters, checkbox checked), `JSON.stringify/JSON.parse` of the form data, object destructuring of the fixture name and email, the spread operator to add a `submissionDate` field, and a closure-based submission counter. All behaviors were verified in the browser console (validation alerts, JSON logging, destructuring, spread, and an incrementing submission count).

Code: `index.html`

```
<!DOCTYPE html>

<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>HW1-Uday Patel</title>
  </head>
  <body>
    <h1>
      Community Sports League Fixtures
    </h1>

    <form id = "fixtureForm">
      <div style="margin-bottom: 20px;">
        <!-- The autofocus will make the cursor focus on this -->
        <label for="fixtureName">Fixture Name:</label>
        <input type="text" id="fixtureName" name="fixtureName"
          placeholder="Enter the names of the teams playing" size = "50" required autofocus>
      </div>

      <div style="margin-bottom: 20px;">
        <label for="teamsPlayers">Teams / Players:</label>
        <input type="text" id="teamsPlayers" name="teamsPlayers"
          placeholder="e.g. Team Milpitas Lions vs Team San Jose Tigers roster" required
          style="width: 500px; height: 50px;">
      </div>

      <div style="margin-bottom: 20px;">
        <label for="submitterEmail">Your Email:</label>
        <input type="email" id="submitterEmail" name="submitterEmail"
          placeholder="e.g. captain@leaguemail.com" required>
      </div>

      <div style="margin-bottom: 20px;">
        <label for="description">Match Details:</label>
        <textarea id="description" name="description"
          placeholder="Describe the match: rules, notes, special instructions..."
          required></textarea>
      </div>

      <div style="margin-bottom: 20px;">
        <label for="category">Sport Type:</label>
        <select id="category" name="category">
          <option value="soccer">Soccer</option>
          <option value="basketball">Basketball</option>
```

```

        <option value="volleyball">Volleyball</option>
        <option value="cricket">Cricket</option>
    </select>
</div>

<div style = "margin-bottom: 20px;">
    <input type="checkbox" id="agreeTerms" name="agreeTerms">
    <label for="agreeTerms">I agree to the terms and conditions.</label>
</div>

<div>
    <button type="submit">Submit Fixture</button>
</div>
</form>

<script src="app.js"></script>
</body>
</html>

```

Code: app.js

```

// Get a reference to the form
const fixtureForm = document.getElementById("fixtureForm");

// Requirement II.5: closure for tracking submission count
function createSubmissionCounter() {
    let count = 0;
    return function() {
        count = count + 1;
        return count;
    };
}

const trackSubmission = createSubmissionCounter();

// Requirement II.1: arrow function to validate description length + checkbox
const validateForm = (description, agreeTerms) => {
    if (description.length <= 25) {
        alert("Match details must be more than 25 characters.");
        return false;
    }
    if (!agreeTerms) {
        alert("You must agree to the terms and conditions.");
        return false;
    }
    return true;
};

// Handle form submission
fixtureForm.addEventListener("submit", function(event) {
    event.preventDefault(); // stop the page from reloading

    const description = document.getElementById("description").value;
    const agreeTerms = document.getElementById("agreeTerms").checked;

    if (!validateForm(description, agreeTerms)) {
        return; // stop here if validation failed
    }
}

```

```
// Requirement II.2: convert form data to JSON string, log it
const formData = {
  fixtureName: document.getElementById("fixtureName").value,
  teamsPlayers: document.getElementById("teamsPlayers").value,
  submitterEmail: document.getElementById("submitterEmail").value,
  description: description,
  category: document.getElementById("category").value,
  agreeTerms: agreeTerms
};

const jsonString = JSON.stringify(formData);
console.log("Form data as JSON string:", jsonString);

// Requirement II.3: parse it back, then destructure
const parsedObject = JSON.parse(jsonString);
const { fixtureName, submitterEmail } = parsedObject;
console.log("Fixture Name:", fixtureName);
console.log("Submitter Email:", submitterEmail);

// Requirement II.4: spread operator to add submissionDate
const updatedObject = { ...parsedObject, submissionDate: new Date().toISOString() };
console.log("Updated object with submission date:", updatedObject);

// Requirement II.5: log the submission count using the closure
const submissionCount = trackSubmission();
console.log("Submission count:", submissionCount);
});
```

Code: Dockerfile

```
# Use Nginx as the base image
FROM nginx:alpine

# Remove default Nginx HTML files
RUN rm -rf /usr/share/nginx/html/*

# Copy frontend files (HTML, JS) into the Nginx web root
COPY web_application/ /usr/share/nginx/html/

# Configure Nginx to listen on PORT_BASE (8871) instead of default 80
RUN sed -i 's/listen\s*80;/listen 8871;/' /etc/nginx/conf.d/default.conf

# Expose PORT_BASE
EXPOSE 8871

# Start Nginx
CMD ["nginx", "-g", "daemon off;"]
```

Browser testing

The form was tested directly in the browser with the DevTools console open, confirming the JSON string produced by `JSON.stringify`, the destructured `fixtureName` and `submitterEmail` values, and the updated object produced by the spread operator (including the added `submissionDate` field):

Community Sports League Fixtures

Fixture Name:

Teams / Players:

Your Email:

Match Details:

Sport Type:

☒ I agree to the terms and conditions.

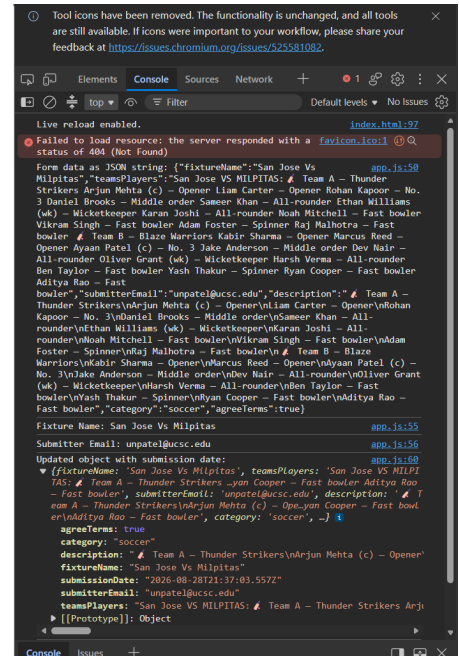


Figure 1: Form filled out in the browser, with the console showing the JSON-stringified form data, destructured `fixtureName/submitterEmail`, and the spread-operator updated object with `submissionDate` added. The closure-based submission counter was also verified separately by submitting the form multiple times and observing the logged count increment each time.

Docker deployment

The app was containerized with an Nginx-based Dockerfile serving on PORT_BASE (8871), built and run locally, and verified in the browser at `localhost:8871`:

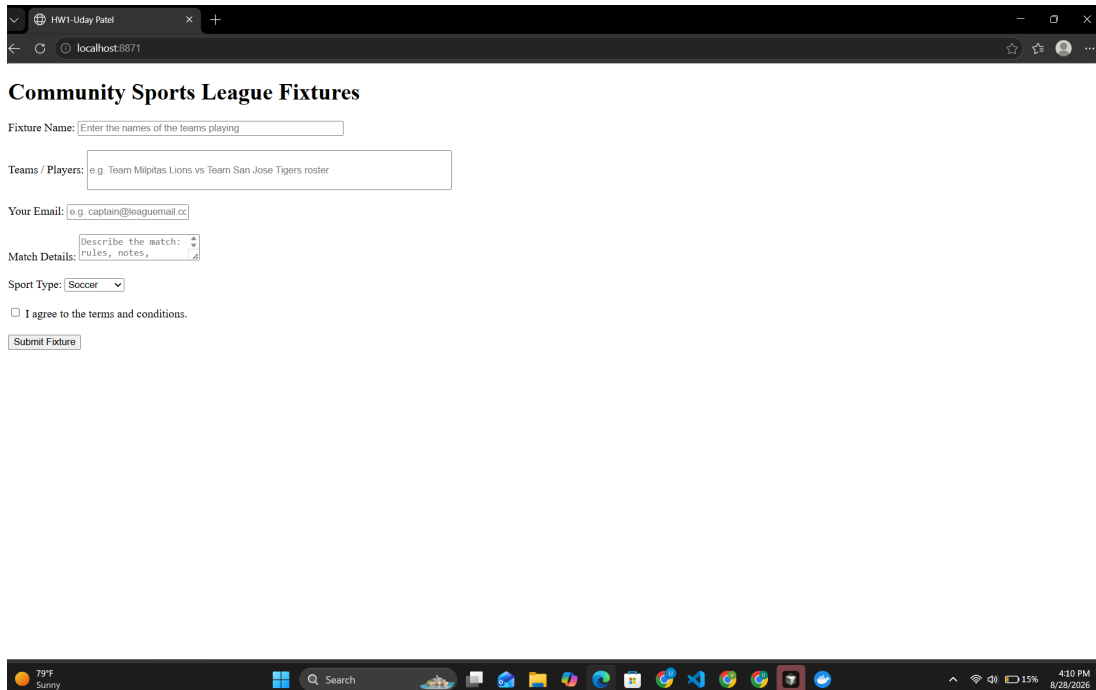


Figure 2: Application running locally via Docker (`localhost:8871`).

AWS ECS deployment

The image was pushed to a private Amazon ECR repository (`my_web_app`), then deployed as a single Fargate task (0.25 vCPU / 0.5 GB) in an ECS service, with a dedicated security group allowing inbound TCP traffic on port 8871 from any source, and a public IP enabled. The running task was verified at its public IP:

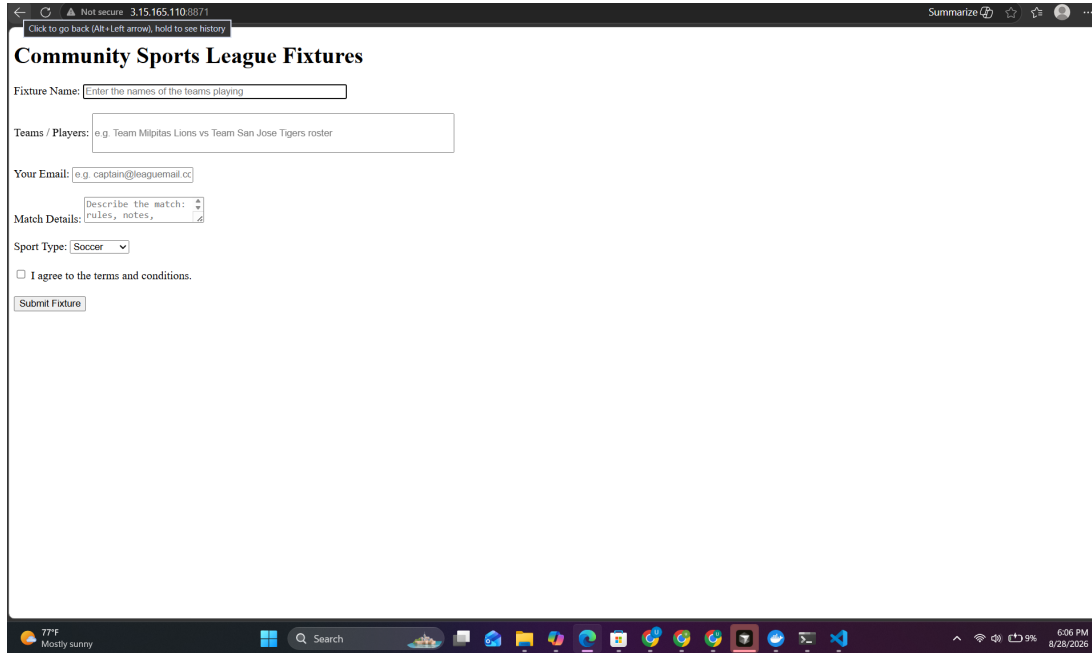


Figure 3: Application running on AWS ECS/Fargate, accessed at its public IP (3.15.165.110:8871).

Part 2 – Agentic AI (Planner / Reviewer / Finalizer)

Two agents (Planner, Reviewer) plus a Finalizer step were built in `agents_demo.py`, using qwen3:8b via Ollama and LangChain. The Planner proposes 3 tags and a ≤ 25 -word summary purely from the input title/content (no hardcoded domain keywords). The Reviewer checks the draft against the same two criteria and either confirms it or corrects it. The Finalizer strips any model reasoning trace and parses the result into strict JSON.

Command used

```
python agents_demo.py
```

Code: agents_demo.py

```
"""
DATA-260 Homework 1 - Part 2: Agentic AI
Two small agents (Planner, Reviewer) plus a finalization step that produce
exactly 3 topical tags and a <=25 word summary from a domain entity's
title/content, using a local LLM served through Ollama.

Domain: Community sports league fixtures (DOMAIN_ID = 7)
Model: qwen3:8b
"""

import json
import re

from langchain_ollama import ChatOllama

# Model client - temperature is intentionally a parameter here since
# Part 3 requires re-running this pipeline at temperature 0.7 and 0.0.
model = ChatOllama(model="qwen3:8b", temperature=0.7)

def run_planner(title, content, llm=None):
    """Planner agent: proposes 3 tags + a <=25 word summary from the input."""
    llm = llm or model
    prompt = f"""You are a Planner agent. Given a title and content, propose:
- exactly 3 short topical tags (lowercase, 1-3 words each)
- a one-sentence summary of at most 25 words

Do not use fixed categories - derive tags and summary purely from the input text.

Title: {title}
Content: {content}

Respond ONLY with valid JSON in this exact format, nothing else:
{{"tags": ["tag1", "tag2", "tag3"], "summary": "your summary here"}}
"""
    response = llm.invoke(prompt)
    return response.content

def run_reviewer(planner_output, title, content, llm=None):
    """Reviewer agent: checks the Planner's draft and corrects it if needed."""
    llm = llm or model
    prompt = f"""You are a Reviewer agent. You will check another AI's draft tags and summary for
quality.
```

```

Original title: {title}
Original content: {content}

Draft to review:
{planner_output}

Check that:
1. There are EXACTLY 3 tags, each 1-3 words, lowercase, genuinely specific to the content (not generic
   ).
2. The summary is ONE sentence, at most 25 words, and accurately reflects the content.

If the draft already satisfies both conditions, return it unchanged.
If not, correct it yourself.

Respond ONLY with valid JSON in this exact format, nothing else:
{"tags": ["tag1", "tag2", "tag3"], "summary": "your summary here"}}
"""
    response = llm.invoke(prompt)
    return response.content

def extract_json(text):
    """Strip any reasoning/thinking blocks and pull out the first JSON object."""
    text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL)
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        raise ValueError(f"No JSON object found in model output:\n{text}")
    return json.loads(match.group())

def finalize(review_output):
    """Finalizer: parses the Reviewer's output into the final clean JSON."""
    parsed = extract_json(review_output)
    return {
        "tags": parsed["tags"],
        "summary": parsed["summary"],
    }

def main():
    title = "San Jose Strikers vs Milpitas Warriors - Rivalry Clash"
    content = (
        "The San Jose Strikers and Milpitas Warriors renew their fierce Bay Area "
        "rivalry this weekend at Cricket Ground South. The two clubs have split "
        "their last six meetings evenly, and this fixture carries extra weight as "
        "both teams sit near the top of the league standings. San Jose's fast "
        "bowlers will look to exploit a green pitch early, while Milpitas counts "
        "on its experienced middle order to chase down any target. Gates open at "
        "9:00 AM with the toss scheduled for 9:30 AM."
    )

    print("=== Planner Output ===")
    planner_raw = run_planner(title, content)
    print(planner_raw)

    print("\n=== Reviewer Output ===")
    reviewer_raw = run_reviewer(planner_raw, title, content)
    print(reviewer_raw)

```

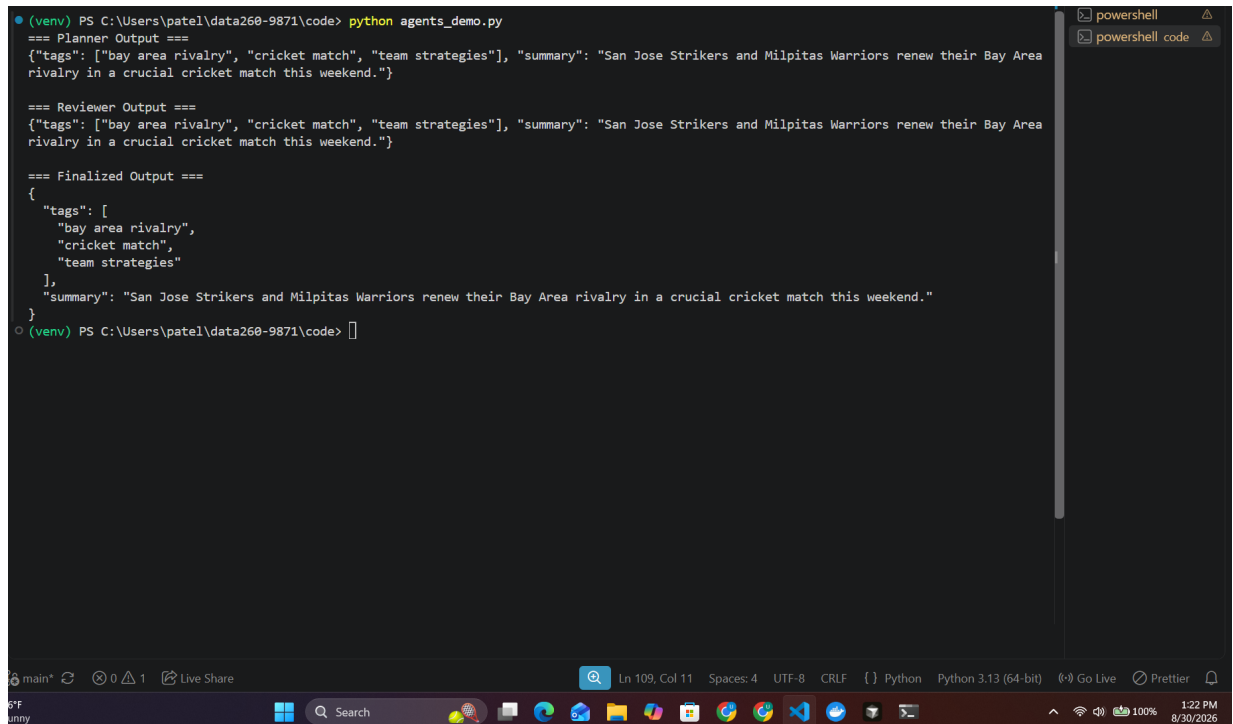
```

print("\n=== Finalized Output ===")
final_result = finalize(reviewer_raw)
print(json.dumps(final_result, indent=2))

if __name__ == "__main__":
    main()

```

Console output (Planner, Reviewer, Finalized)



```

(venv) PS C:\Users\patel\data260-9871\code> python agents_demo.py
=== Planner Output ===
{"tags": ["bay area rivalry", "cricket match", "team strategies"], "summary": "San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend."}

=== Reviewer Output ===
{"tags": ["bay area rivalry", "cricket match", "team strategies"], "summary": "San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend."}

=== Finalized Output ===
{
  "tags": [
    "bay area rivalry",
    "cricket match",
    "team strategies"
  ],
  "summary": "San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend."
}
(venv) PS C:\Users\patel\data260-9871\code>

```

Figure 4: Terminal output of `python agents_demo.py`, showing the Planner, Reviewer, and Finalized JSON output.

Short answers

Q1. The 3 final tags: “bay area rivalry”, “cricket match”, “team strategies”

Q2. The final summary: “San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend.” (18 words)

Q3. Did the Reviewer change anything? No. The Reviewer received the Planner’s draft and returned it unchanged, since the draft already had exactly 3 specific tags and an 18-word summary that accurately reflected the input – both conditions the Reviewer was instructed to check were already satisfied.

Explanation of each step

Planner: receives the raw title/content and is prompted to propose exactly 3 tags and a one-sentence, ≤ 25 -word summary, deriving both purely from the text rather than fixed categories.

Reviewer: receives the Planner’s draft (plus the original title/content) and checks it against the same two criteria (tag count/specificity, summary length/accuracy) – either confirming it unchanged or correcting it.

Finalizer: takes the Reviewer’s raw text output, strips any internal reasoning trace the model may have emitted (qwen3:8b sometimes wraps output in a reasoning block), extracts the JSON object, and returns a clean Python dict with exactly the **tags** and **summary** keys required by the assignment.

Part 3 – Measuring Non-Determinism

Using a fixed input (saved at `reports/hw01/cases/nondeterminism_input.json`), the full Planner-Reviewer-Finalizer pipeline was run 20 times at temperature 0.7 and 20 times at temperature 0.0, recording the final tags and latency for each run.

Command used

```
python measure_nondeterminism.py
```

Code: `measure_nondeterminism.py`

```
"""
DATA-260 Homework 1 - Part 3: Measuring Non-Determinism
Runs the Planner -> Reviewer -> Finalizer pipeline on a single fixed input
20 times at temperature 0.7 and 20 times at temperature 0.0, records the
final tags and latency for each run, saves the raw results, and computes
summary statistics (distinct tag sets, tags in all runs, tags in exactly
one run, latency p50/p95/p99).
"""

import csv
import json
import statistics
import time
from pathlib import Path

from langchain_ollama import ChatOllama

from agents_demo import run_planner, run_reviewer, finalize

# ---- Paths -----
THIS_DIR = Path(__file__).resolve().parent
REPO_ROOT = THIS_DIR.parent
CASES_DIR = REPO_ROOT / "reports" / "hw01" / "cases"
RAW_DIR = REPO_ROOT / "reports" / "hw01" / "raw"
RAW_DIR.mkdir(parents=True, exist_ok=True)

INPUT_FILE = CASES_DIR / "nondeterminism_input.json"
RUNS_PER_TEMPERATURE = 20
TEMPERATURES = [0.7, 0.0]

def load_fixed_input():
    with open(INPUT_FILE, "r", encoding="utf-8") as f:
        data = json.load(f)
    return data["title"], data["content"]

def run_once(title, content, temperature):
    """Run the full pipeline once at the given temperature. Returns (tags, latency_ms)."""
    llm = ChatOllama(model="qwen3:8b", temperature=temperature)

    start = time.perf_counter()
    planner_raw = run_planner(title, content, llm=llm)
    reviewer_raw = run_reviewer(planner_raw, title, content, llm=llm)
    result = finalize(reviewer_raw)
    end = time.perf_counter()
```

```

latency_ms = (end - start) * 1000
return result["tags"], latency_ms

def percentile(values, pct):
    """Simple percentile using linear interpolation, no numpy dependency."""
    if not values:
        return None
    values_sorted = sorted(values)
    k = (len(values_sorted) - 1) * (pct / 100)
    f = int(k)
    c = min(f + 1, len(values_sorted) - 1)
    if f == c:
        return values_sorted[f]
    return values_sorted[f] + (values_sorted[c] - values_sorted[f]) * (k - f)

def summarize(runs):
    """runs: list of (tags_list, latency_ms). Returns a stats dict."""
    tag_sets = [tuple(sorted(t)) for t, _ in runs]
    distinct_tag_sets = len(set(tag_sets))

    tag_run_counts = {}
    for tags, _ in runs:
        for tag in set(tags):
            tag_run_counts[tag] = tag_run_counts.get(tag, 0) + 1

    n_runs = len(runs)
    tags_in_all_runs = sorted([t for t, c in tag_run_counts.items() if c == n_runs])
    tags_in_exactly_one_run = sorted([t for t, c in tag_run_counts.items() if c == 1])

    latencies = [lat for _, lat in runs]

    return {
        "distinct_tag_sets": distinct_tag_sets,
        "tags_in_all_runs": tags_in_all_runs,
        "tags_in_exactly_one_run": tags_in_exactly_one_run,
        "latency_p50_ms": round(percentile(latencies, 50), 1),
        "latency_p95_ms": round(percentile(latencies, 95), 1),
        "latency_p99_ms": round(percentile(latencies, 99), 1),
        "latency_mean_ms": round(statistics.mean(latencies), 1),
    }

def main():
    title, content = load_fixed_input()
    print(f"Fixed input title: {title}\n")

    all_results = {}

    for temperature in TEMPERATURES:
        print(f"=== Running {RUNS_PER_TEMPERATURE} runs at temperature {temperature} ===")
        runs = []
        raw_rows = []

        for i in range(1, RUNS_PER_TEMPERATURE + 1):
            tags, latency_ms = run_once(title, content, temperature)
            runs.append((tags, latency_ms))
            raw_rows.append({

```

```

        "run": i,
        "temperature": temperature,
        "tags": tags,
        "latency_ms": round(latency_ms, 1),
    })
    print(f" run {i:2d}: tags={tags} latency={latency_ms:.1f}ms")

temp_label = str(temperature).replace(".", "_")
raw_json_path = RAW_DIR / f"nondeterminism_temp_{temp_label}.json"
raw_csv_path = RAW_DIR / f"nondeterminism_temp_{temp_label}.csv"

with open(raw_json_path, "w", encoding="utf-8") as f:
    json.dump(raw_rows, f, indent=2)

with open(raw_csv_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["run", "temperature", "tags", "latency_ms"])
    writer.writeheader()
    for row in raw_rows:
        writer.writerow({**row, "tags": json.dumps(row["tags"])})

stats = summarize(runs)
all_results[temperature] = stats

print(f"\n Summary for temperature {temperature}:")
print(json.dumps(stats, indent=4))
print()

summary_path = RAW_DIR / "nondeterminism_summary.json"
with open(summary_path, "w", encoding="utf-8") as f:
    json.dump({str(k): v for k, v in all_results.items()}, f, indent=2)

print(f"Raw per-run results and summary saved to {RAW_DIR}")

if __name__ == "__main__":
    main()

```

Console output

Fixed input title: San Jose Strikers vs Milpitas Warriors - Rivalry Clash

```

=== Running 20 runs at temperature 0.7 ===
run 1: tags=['bay area rivalry', 'cricket match', 'league standings'] latency=51873.7ms
run 2: tags=['bay area rivalry', 'cricket match', 'rivalry clash'] latency=66895.9ms
run 3: tags=['bay area rivalry', 'cricket match', 'rivalry clash'] latency=63997.8ms
run 4: tags=['bay area rivalry', 'cricket match', 'league title race'] latency=24763.7ms
run 5: tags=['bay area rivalry', 'cricket match', 'top league teams'] latency=27106.0ms
run 6: tags=['bay area rivalry', 'cricket match', 'cricket rivalry'] latency=71703.0ms
run 7: tags=['bay area rivalry', 'cricket match', 'cricket rivalry'] latency=81831.6ms
run 8: tags=['bay area rivalry', 'cricket match', 'rivalry clash'] latency=49852.0ms
run 9: tags=['bay area rivalry', 'cricket match', 'team strategies'] latency=36306.2ms
run 10: tags=['bay area rivalry', 'cricket match', 'game strategy'] latency=32841.4ms
run 11: tags=['bay area rivalry', 'cricket match', 'league standings'] latency=38255.9ms
run 12: tags=['bay area rivalry', 'cricket match', 'strategic play'] latency=30661.7ms
run 13: tags=['bay area rivalry', 'cricket match', 'team standings'] latency=50526.6ms
run 14: tags=['bay area rivalry', 'cricket match', 'rivalry clash'] latency=40337.8ms
run 15: tags=['bay area rivalry', 'cricket match', 'rivalry clash'] latency=60444.4ms
run 16: tags=['bay area rivalry', 'cricket match', 'fast bowling'] latency=33027.8ms
run 17: tags=['bay area rivalry', 'cricket match', 'rivalry clash'] latency=59641.7ms

```

```

run 18: tags=['rivalry clash', 'bay area', 'cricket match'] latency=33842.1ms
run 19: tags=['bay area rivalry', 'green pitch', 'middle order'] latency=64918.8ms
run 20: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48713.0ms

```

Summary for temperature 0.7:

```

{
  "distinct_tag_sets": 13,
  "tags_in_all_runs": [],
  "tags_in_exactly_one_run": [
    "bay area", "fast bowling", "game strategy", "green pitch",
    "league title race", "middle order", "strategic play",
    "team rivalry", "team standings", "team strategies",
    "top league teams"
  ],
  "latency_p50_ms": 49282.5,
  "latency_p95_ms": 72209.5,
  "latency_p99_ms": 79907.2,
  "latency_mean_ms": 48377.1
}

```

=== Running 20 runs at temperature 0.0 ===

```

run 1: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=49128.6ms
run 2: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=49081.0ms
run 3: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=49044.7ms
run 4: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48734.0ms
run 5: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48707.0ms
run 6: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48669.5ms
run 7: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48643.6ms
run 8: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48615.6ms
run 9: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48676.3ms
run 10: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48658.7ms
run 11: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48843.2ms
run 12: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=49805.7ms
run 13: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48682.8ms
run 14: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48631.9ms
run 15: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48777.7ms
run 16: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=49360.7ms
run 17: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48680.1ms
run 18: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48881.2ms
run 19: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48824.1ms
run 20: tags=['bay area rivalry', 'cricket match', 'team rivalry'] latency=48654.7ms

```

Summary for temperature 0.0:

```

{
  "distinct_tag_sets": 1,
  "tags_in_all_runs": [
    "bay area rivalry", "cricket match", "team rivalry"
  ],
  "tags_in_exactly_one_run": [],
  "latency_p50_ms": 48720.5,
  "latency_p95_ms": 49382.9,
  "latency_p99_ms": 49721.1,
  "latency_mean_ms": 48855.1
}

```

Raw per-run results and summary saved to
C:\Users\patel\data260-9871\reports\hw01\raw

Tag Set Variation

Metric	Temp 0.7	Temp 0.0
Distinct tag sets	13	1
Tags in all 20 runs	(none)	bay area rivalry, cricket match, team rivalry
Tags in exactly 1 run	bay area, fast bowling, game strategy, green pitch, league title race, middle order, strategic play, team rivalry, team standings, team strategies, top league teams	(none)

Latency

Metric	Temp 0.7	Temp 0.0
Latency p50 (ms)	49282.5	48720.5
Latency p95 (ms)	72209.5	49382.9
Latency p99 (ms)	79907.2	49721.1

Explanation

At temperature 0.0, the pipeline produced identical tags across all 20 runs, confirming qwen3:8b is effectively deterministic at this setting on this hardware. At temperature 0.7, only 13 of 20 runs produced a unique tag combination, and no single tag appeared in every run – even “bay area rivalry” and “cricket match,” which appeared in 19/20 runs, each had one outlier run. Latency was also far more consistent at temp 0.0 (a roughly 1-second range) than at temp 0.7 (a roughly 57-second range), suggesting randomness affects not just word choice but how much the model explores before settling on an answer.

Two users sending identical input at temperature 0.0 would see exactly the same tags and summary every time – useful when consistency matters, such as an automated compliance or audit classification system where the same input must always yield the same category for legal defensibility. Two users sending identical input at temperature 0.7 would likely see different (though related) tags – acceptable, and even desirable, for a creative tag-suggestion feature where variety helps users discover different angles on their content, but not acceptable for a system that needs reproducible, auditable output.

Part 4 – Model Client and Token Accounting

A reusable model-adapter class (`ModelClient`) was built at `src/model_client.py`, exposing a stable `complete(messages, tools=None)` interface. All model calls in `hw1_client.py` go through this adapter, which tracks conversation history and cumulative token counts internally. `hw1_client.py` is a small CLI that loads `AGENT.md` as the system prompt (instructing the model to act as a strict, bullet-only code reviewer) and supports a `/stats` command.

Command used

```
python hw1_client.py
```

Code: `src/model_client.py`

```
"""
DATA-260 Homework 1 - Part 4: Model Client and Token Accounting
A reusable model-adapter module. All model calls in this project should go
through complete(messages, tools=None) rather than calling the underlying
LangChain/Ollama client directly - this keeps one stable interface, and lets
us track token usage and conversation history in a single place.
"""

import json
import re

from langchain_ollama import ChatOllama

def strip_thinking(text):
    """Remove <think>...</think> reasoning blocks some Ollama models emit,
    so neither the printed output nor the stored history is polluted with
    the model's internal reasoning trace."""
    return re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL).strip()

class ModelClient:
    """Wraps a local Ollama model behind a stable complete() interface,
    tracking conversation history and cumulative token usage."""

    def __init__(self, model="qwen3:8b", temperature=0.7):
        self._llm = ChatOllama(model=model, temperature=temperature)
        self.history = [] # list of {"role": ..., "content": ...} dicts

        self.turn_count = 0
        self.cumulative_input_tokens = 0
        self.cumulative_output_tokens = 0

    def complete(self, messages, tools=None):
        """Send 'messages' (a list of {"role", "content"} dicts) to the model
        and return the assistant's reply as a string. 'tools' is accepted for
        interface stability but is not used by this simple adapter yet.

        This also appends both the outgoing user message(s) and the model's
        reply to self.history, and updates token/turn counters.
        """
        lc_messages = [(m["role"], m["content"]) for m in messages]

        response = self._llm.invoke(lc_messages)
```

```

        clean_content = strip_thinking(response.content)

    for m in messages:
        if m not in self.history:
            self.history.append(m)
    self.history.append({"role": "assistant", "content": clean_content})

    usage = getattr(response, "usage_metadata", None) or {}
    input_tokens = usage.get("input_tokens", 0)
    output_tokens = usage.get("output_tokens", 0)
    total_tokens = usage.get("total_tokens", input_tokens + output_tokens)

    self.turn_count += 1
    self.cumulative_input_tokens += input_tokens
    self.cumulative_output_tokens += output_tokens

    print(f"[turn {self.turn_count}] input_tokens={input_tokens} "
          f"output_tokens={output_tokens} total_tokens={total_tokens}")

    return clean_content

def stats(self):
    """Return a dict of current stats: turn count, cumulative token
    counts, and serialized conversation-history length. Does NOT alter
    self.history."""
    history_json = json.dumps(self.history)
    return {
        "turn_count": self.turn_count,
        "cumulative_input_tokens": self.cumulative_input_tokens,
        "cumulative_output_tokens": self.cumulative_output_tokens,
        "cumulative_total_tokens": (
            self.cumulative_input_tokens + self.cumulative_output_tokens
        ),
        "history_length_chars": len(history_json),
    }

def print_final_summary(self):
    """Print cumulative totals - intended to be called on exit."""
    print("\n=== Final Summary ===")
    print(f"Total turns: {self.turn_count}")
    print(f"Cumulative input tokens: {self.cumulative_input_tokens}")
    print(f"Cumulative output tokens: {self.cumulative_output_tokens}")
    print(f"Cumulative total tokens: "
          f"{self.cumulative_input_tokens + self.cumulative_output_tokens}")

```

Code: hw1_client.py

```

"""
DATA-260 Homework 1 - Part 4: hw1_client.py
A small command-line demo that imports the reusable model-adapter
(src/model_client.py) and runs an interactive chat loop. Supports a
/stats command that shows turn count, cumulative token counts, and
serialized conversation-history length, without altering the history.

Run with: python hw1_client.py
Type /stats to see current stats, /exit to quit.
"""

import sys

```

```

from pathlib import Path

# src/ is a sibling of code/ at the repo root - add it to the import path
REPO_ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(REPO_ROOT / "src"))

from model_client import ModelClient # noqa: E402 (import after path setup)

SYSTEM_PROMPT_PATH = REPO_ROOT / "AGENT.md"

def load_system_prompt():
    """Load AGENT.md as the system prompt, if it exists."""
    if SYSTEM_PROMPT_PATH.exists():
        return SYSTEM_PROMPT_PATH.read_text(encoding="utf-8")
    return "You are a helpful assistant."

def print_stats(client):
    stats = client.stats()
    print("\n--- /stats ---")
    print(f"Turn count: {stats['turn_count']}")
    print(f"Cumulative input tokens: {stats['cumulative_input_tokens']}")
    print(f"Cumulative output tokens: {stats['cumulative_output_tokens']}")
    print(f"Cumulative total tokens: {stats['cumulative_total_tokens']}")
    print(f"Serialized history length (chars): {stats['history_length_chars']}")
    print("-----\n")

def main():
    client = ModelClient(model="qwen3:8b", temperature=0.7)
    system_prompt = load_system_prompt()

    print("DATA-260 HW1 - hw1_client.py")
    print("Type your message, /stats to see stats, or /exit to quit.\n")

    while True:
        user_input = input("You: ").strip()

        if not user_input:
            continue

        if user_input == "/exit":
            client.print_final_summary()
            break

        if user_input == "/stats":
            print_stats(client)
            continue

        # Build the message list: system prompt + full history so far + new user message
        messages = [{"role": "system", "content": system_prompt}]
        messages += client.history
        messages.append({"role": "user", "content": user_input})

        reply = client.complete(messages)
        print(f"\nAssistant: {reply}\n")

```

```
if __name__ == "__main__":  
    main()
```

5-turn conversation, with /stats after turn 3 and turn 5

```
You: Review this code: def divide(a, b): return a / b  
[turn 1] input_tokens=127 output_tokens=384 total_tokens=511  
Assistant: - No handling for division by zero (b == 0)  
- No type validation for inputs  
- Uses floating-point division (/) instead of integer division (//) if intended  
- No error handling for non-numeric inputs  
  
You: Review this code: def add(a, b): return a+b  
[turn 2] input_tokens=294 output_tokens=606 total_tokens=900  
Assistant: - No type validation for inputs (e.g., non-numeric types like strings)  
- No error handling for invalid operations (e.g., adding incompatible types)  
- No documentation or clarity on expected input types  
- Potential silent failure if inputs are not numbers  
  
You: What did the first function I asked you to review do?  
[turn 3] input_tokens=367 output_tokens=482 total_tokens=849  
Assistant: The first function you asked me to review was divide(a, b),  
which performs floating-point division of a by b and returns the  
result. It lacks error handling for division by zero, type  
validation, and non-numeric inputs.  
  
You: /stats  
--- /stats ---  
Turn count: 3  
Cumulative input tokens: 788  
Cumulative output tokens: 1472  
Cumulative total tokens: 2260  
Serialized history length (chars): 1541  
-----  
  
You: Thanks, that's helpful  
[turn 4] input_tokens=434 output_tokens=270 total_tokens=704  
Assistant: You're welcome! If you have more code to review or  
questions, feel free to ask.  
  
You: Can you review one more: def subtract(a, b): return a - b  
[turn 5] input_tokens=482 output_tokens=379 total_tokens=861  
Assistant: - No type validation for inputs (e.g., non-numeric types like strings)  
- No error handling for invalid operations (e.g., subtracting incompatible types)  
- No documentation or clarity on expected input types  
- Potential silent failure if inputs are not numbers  
  
You: /stats  
--- /stats ---  
Turn count: 5  
Cumulative input tokens: 1704  
Cumulative output tokens: 2121  
Cumulative total tokens: 3825  
Serialized history length (chars): 2123  
-----  
  
You: /exit  
=== Final Summary ===  
Total turns: 5
```

```
Cumulative input tokens: 1704
Cumulative output tokens: 2121
Cumulative total tokens: 3825
```

Report and README answers

Why is prior conversation context resent with every turn? LLMs are stateless between API calls – the model has no memory of earlier messages once a request finishes. The only way it can “remember” earlier turns is if the full history is explicitly included in the current request. This is demonstrated directly by turn 3 above: the model correctly recalled the exact `divide(a, b)` function from turn 1 only because that entire exchange was resent as part of turn 3’s request.

How is a system prompt different from a user message? A system prompt sets the model’s behavior, role, or constraints for the whole conversation – instructions about how to respond, not content to respond to. A user message is conversational content the model reacts to. Here, `AGENT.md` is sent as the system role on every turn, and its “bullet-only code reviewer” instruction shaped every reply even though no individual user message asked for bullets.

Why do input tokens grow over a conversation? Because the full history is resent every turn, each new request contains strictly more content than the last (system prompt + all previous turns + the new message). This is visible above: turn 1 had 127 input tokens, but by turn 5 input tokens had grown to 482 – nearly 4x – purely because turns 1–4’s messages and replies were now all carried along inside turn 5’s request.

What eventually limits that growth? The model’s context window – a fixed maximum number of tokens (input and output combined) a given model can process in a single request. Once the resent history plus the new message would exceed that limit, older messages must be truncated or summarized, or the request fails. `hw1_client.py` as written never trims history, so in a long enough conversation it would eventually hit this ceiling.